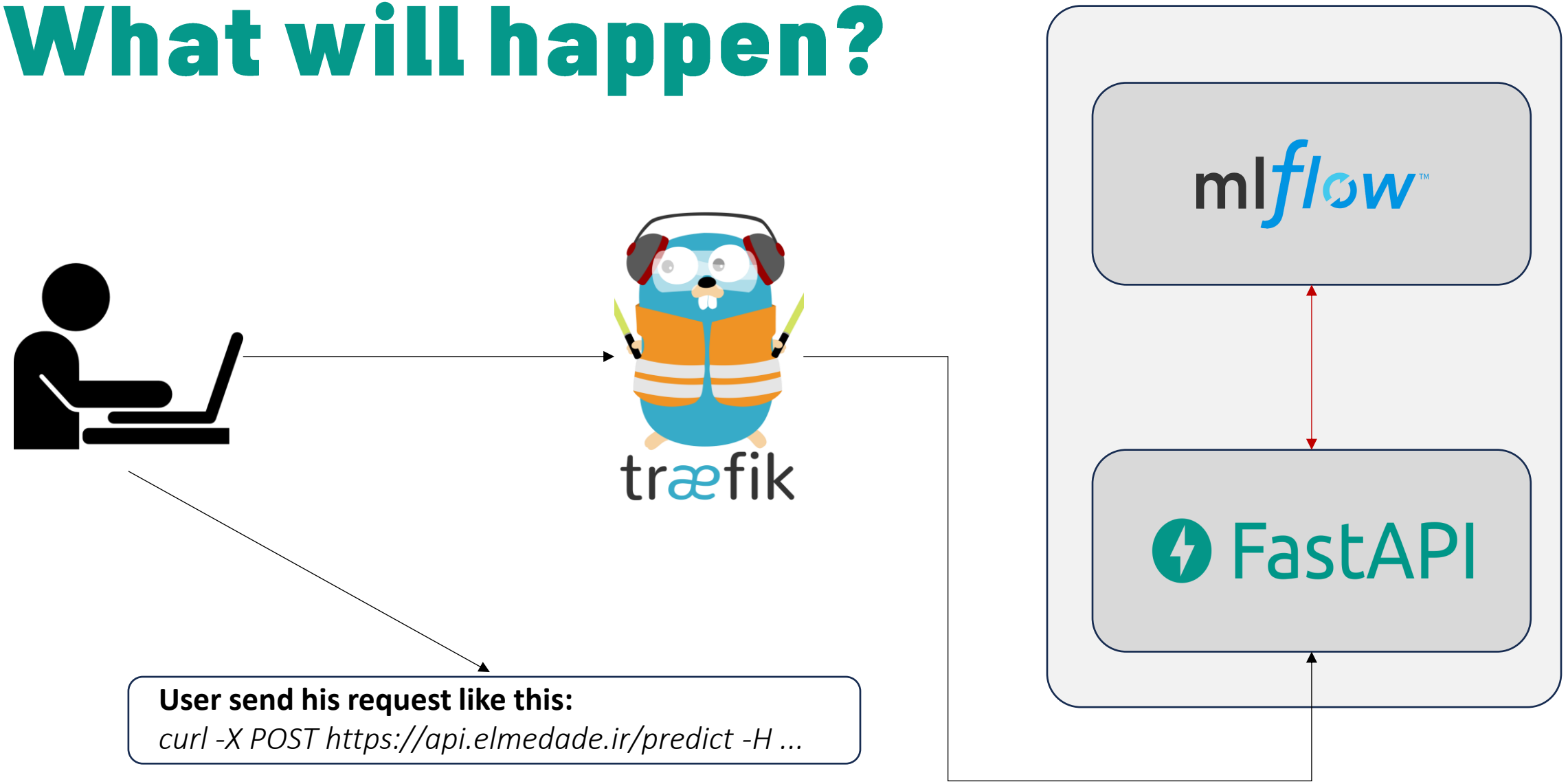
FastAPI for Model Serving Layer

Mohammad Fozouni (Ph.D.)

MLSECOPS IN ACTION COURSE

What will happen?



The most useful question to continue working on the AI/ML solution is ***how costly are wrong predictions?***
Answering that question will define the feasibility of the ML project.



این API که ساختیم، برای محیط عملیاتی مناسب نیست
و ابتدا باید سعی کنیم سوراخ سمبه‌هاش رو ببندیم.



1. **Shape:** Reject malformed input **before** the model. Stops unknown fields & wrong length.
2. **Values + no leak:** Stop NaN/Infinity/out-of-range **before** the model (fixes 500 crash).
3. **Identity and Authentication:** Only authenticated callers reach the model.
4. **Authorization:** Implement scopes; valid token ≠ allowed action.
5. **Availability and Rate-limiting**: Stop floods and model-extraction. Caps cost per request and per minute.
6. **Polish more our API:** security headers, docs off, safe global 500, /health.



1. **Shape:** Reject malformed input **before** the model. Stops unknown fields & wrong length.
2. **Values + no leak:** Stop NaN/Infinity/out-of-range **before** the model (fixes 500 crash).
3. **Identity and Authentication:** Only authenticated callers reach the model.
4. **Authorization:** Implement scopes; valid token ≠ allowed action.
5. **Availability and Rate-limiting**: Stop floods and model-extraction. Caps cost per request and per minute.
6. **Polish more our API:** security headers, docs off, safe global 500, /health.

هدف اینه که تا حد امکان، یک سرویس پایدار (روزنه‌های نفوذ باید بسته شوند) و کاربرپسند ارائه نمائیم.

FastAPI

1. **Shape:** Reject malformed input **before** the model. Stops unknown fields & wrong length.
2. **Values + no leak:** Stop NaN/Infinity/out-of-range **before** the model (fixes 500 crash).
3. **Identity and Authentication:** Only authenticated callers reach the model.
4. **Authorization:** Implement scopes; valid token ≠ allowed action.
5. **Availability and Rate-limiting**: Stop floods and model-extraction. Caps cost per request and per minute.
6. **Polish more our API:** security headers, docs off, safe global 500, /health.

هدف اینه که تا حد امکان، یک سرویس پایدار (روزنه‌های نفوذ باید بسته شوند) و کاربرپسند ارائه نمائیم.

تمام موارد فوق رو با کد مناسب اجرا می‌کنیم. پس در ویدیوی بعدی، تنها کدهای پایتونی خواهیم داشت.

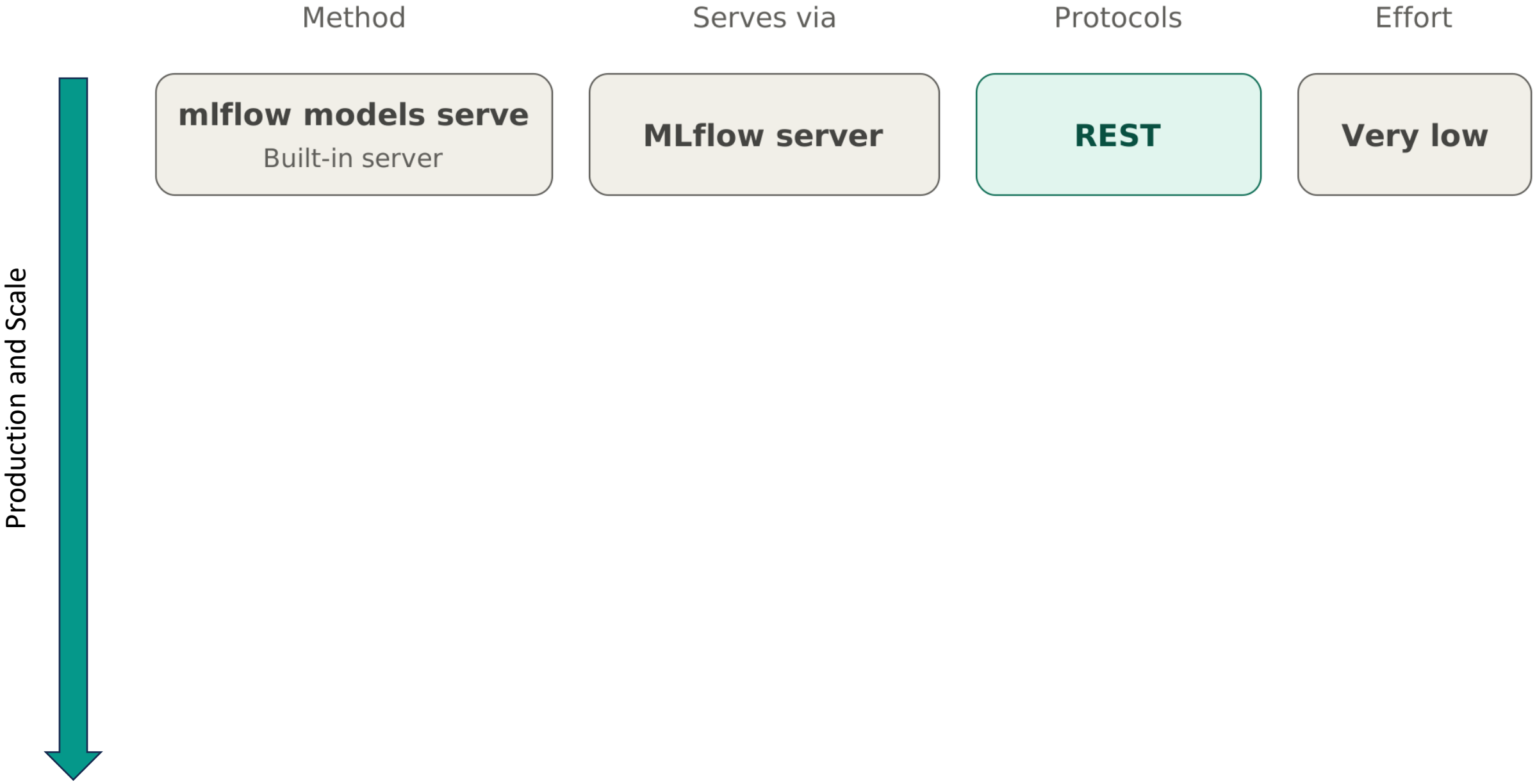


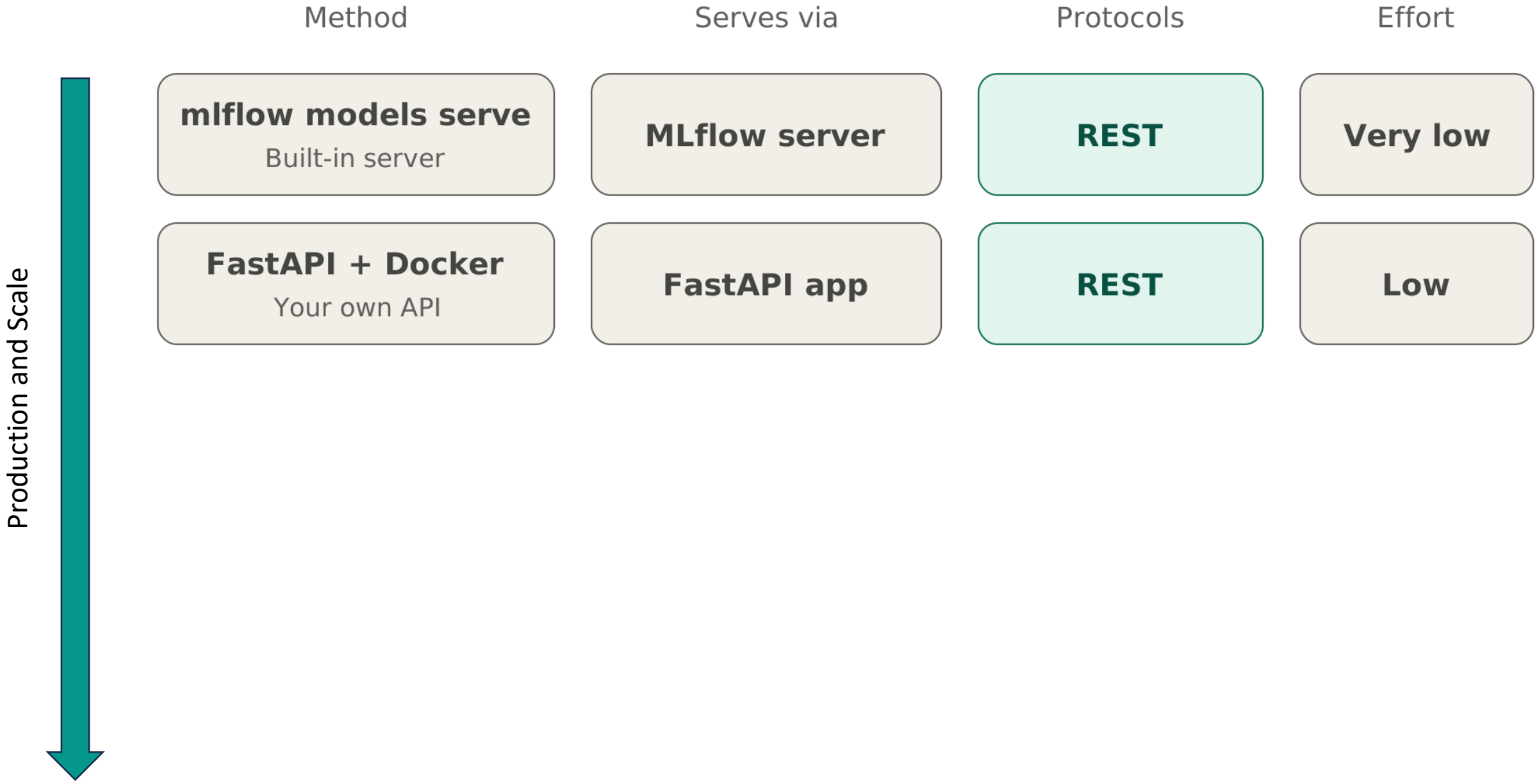


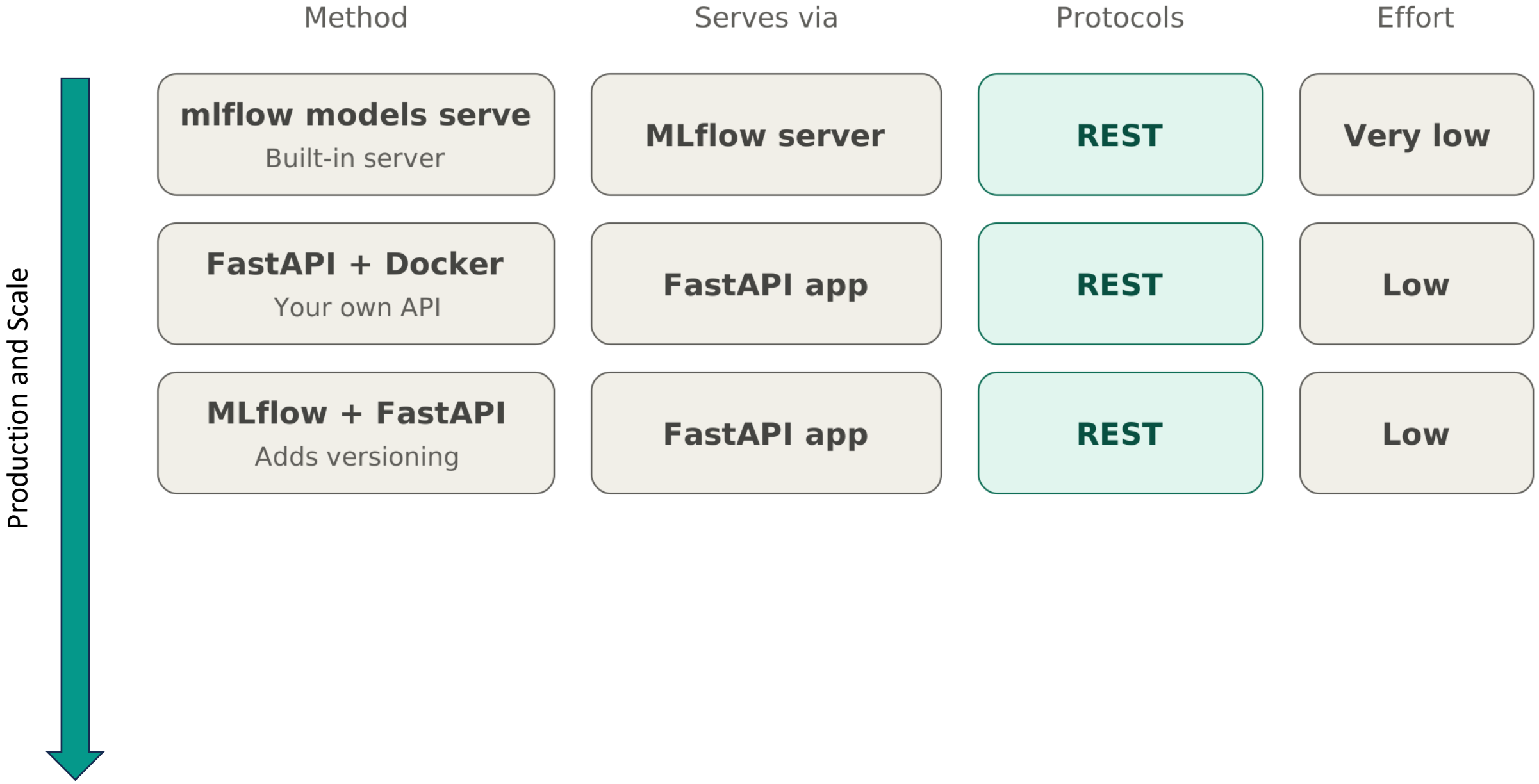
Separating the ML model development from the process that delivers it, lowers quality, transparency, and agility of the whole intelligent software.

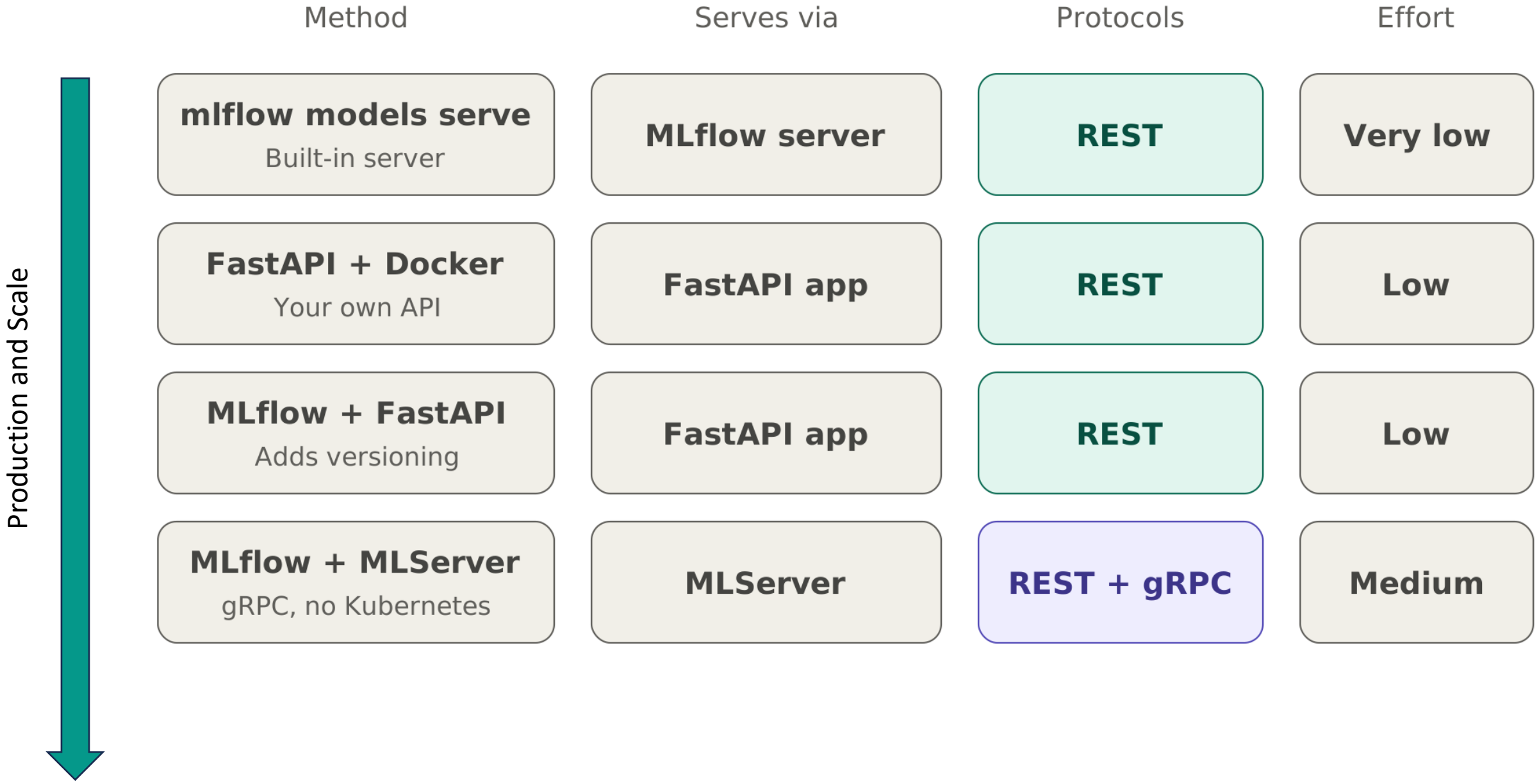
توسعه‌ی مدل‌های یادگیری ماشین و نحوه‌ی رساندن آنها به محیط‌های عملیاتی را نمی‌شود زیاد از یکدیگر جدا کرد. با تفکیک این دو فرایند، کیفیت کلی سیستم کاهش می‌یابد.

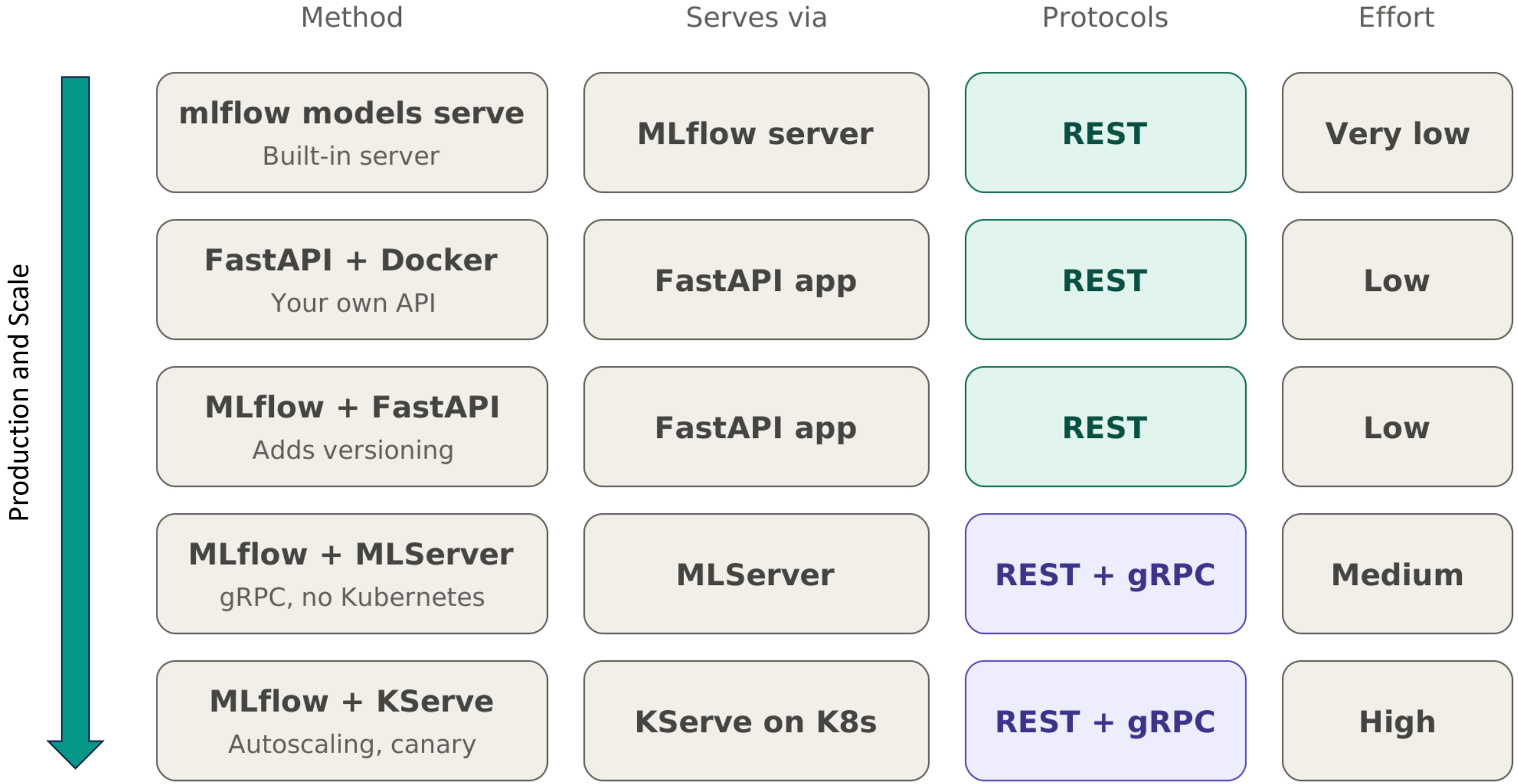
All the current model serving patterns


Method
Serves via
Protocols
Effort
mlflow models serve
Built-in server
MLflow server
REST
Very low
Production and Scale


Method
Serves via
Protocols
Effort
mlflow models serve
Built-in server
MLflow server
REST
Very low
FastAPI + Docker
Your own API
FastAPI app
REST
Low
Production and Scale


Method
Serves via
Protocols
Effort
Production and Scale
mlflow models serve
Built-in server
MLflow server
REST
Very low
FastAPI + Docker
Your own API
FastAPI app
REST
Low
MLflow + FastAPI
Adds versioning
FastAPI app
REST
Low


Method
Serves via
Protocols
Effort
Production and Scale
mlflow models serve
Built-in server
MLflow server
REST
Very low
FastAPI + Docker
Your own API
FastAPI app
REST
Low
MLflow + FastAPI
Adds versioning
FastAPI app
REST
Low
MLflow + MLServer
gRPC, no Kubernetes
MLServer
REST + gRPC
Medium


Method
Serves via
Protocols
Effort
Production and Scale
mlflow models serve
Built-in server
MLflow server
REST
Very low
FastAPI + Docker
Your own API
FastAPI app
REST
Low
MLflow + FastAPI
Adds versioning
FastAPI app
REST
Low
MLflow + MLServer
gRPC, no Kubernetes
MLServer
REST + gRPC
Medium
MLflow + KServe
Autoscaling, canary
KServe on K8s
REST + gRPC
High

Method	Serves via	Protocols	Effort
mlflow models serve Built-in server	**MLflow server**	**REST**	**Very low**
FastAPI + Docker Your own API	**FastAPI app**	**REST**	**Low**
MLflow + FastAPI Adds versioning	**FastAPI app**	**REST**	**Low**
MLflow + MLServer gRPC, no Kubernetes	**MLServer**	**REST + gRPC**	**Medium**
MLflow + KServe Autoscaling, canary	**KServe on K8s**	**REST + gRPC**	**High**



Now Let’s go for practical stuffs and million lines of Python codes 😎